UBO Robotics Lab · Student Tutorial

AutoNateAI

Wire, Code, And Drive

A first robotics loop: circuit wiring, firmware behavior, and game simulation

Robot motion starts with small electrical decisions. A controller pin turns on or off. A motor driver reads that signal. The driver pushes current through a motor. The game reads the robot state and turns it into movement on the screen.

In this lab, you will build that chain from the beginning. You will wire a controller to a motor driver, write firmware that behaves like real robot code, run the circuit in the simulator, and connect the same behavior to the driving screen.

Beginner idea: HIGH and LOW

A digital pin is like a tiny switch your code can control. `HIGH` means the pin is driven toward the board's positive voltage. On an Arduino Uno, that is usually about `5V`. `LOW` means the pin is driven toward ground, which is `0V`. A motor driver reads those voltage levels and uses them to decide which way to push current through the motor load.

The Build Goal

You are building the first complete robotics loop:

Wire the circuit -> Write firmware -> Drive the simulation

The board pins are not decorations. They are control signals. The motor driver turns those small control signals into output states that can move a motor load.

What success looks like

By the end, `forward`, `reverse`, `pivot`, and `stop` should feel like electrical states you can inspect, not just game commands.

1. Start Inside The UBO Lab

Open `https://ubo.autonateai.com/`, start a session, and choose `Guest run` if you do not need a saved login. From the dashboard, open `Wiring lab`.

You should see the UBO shell around the embedded Velxio editor. The lesson happens in this app because the same portal can log the wiring, coding, driving, and reporting data later.

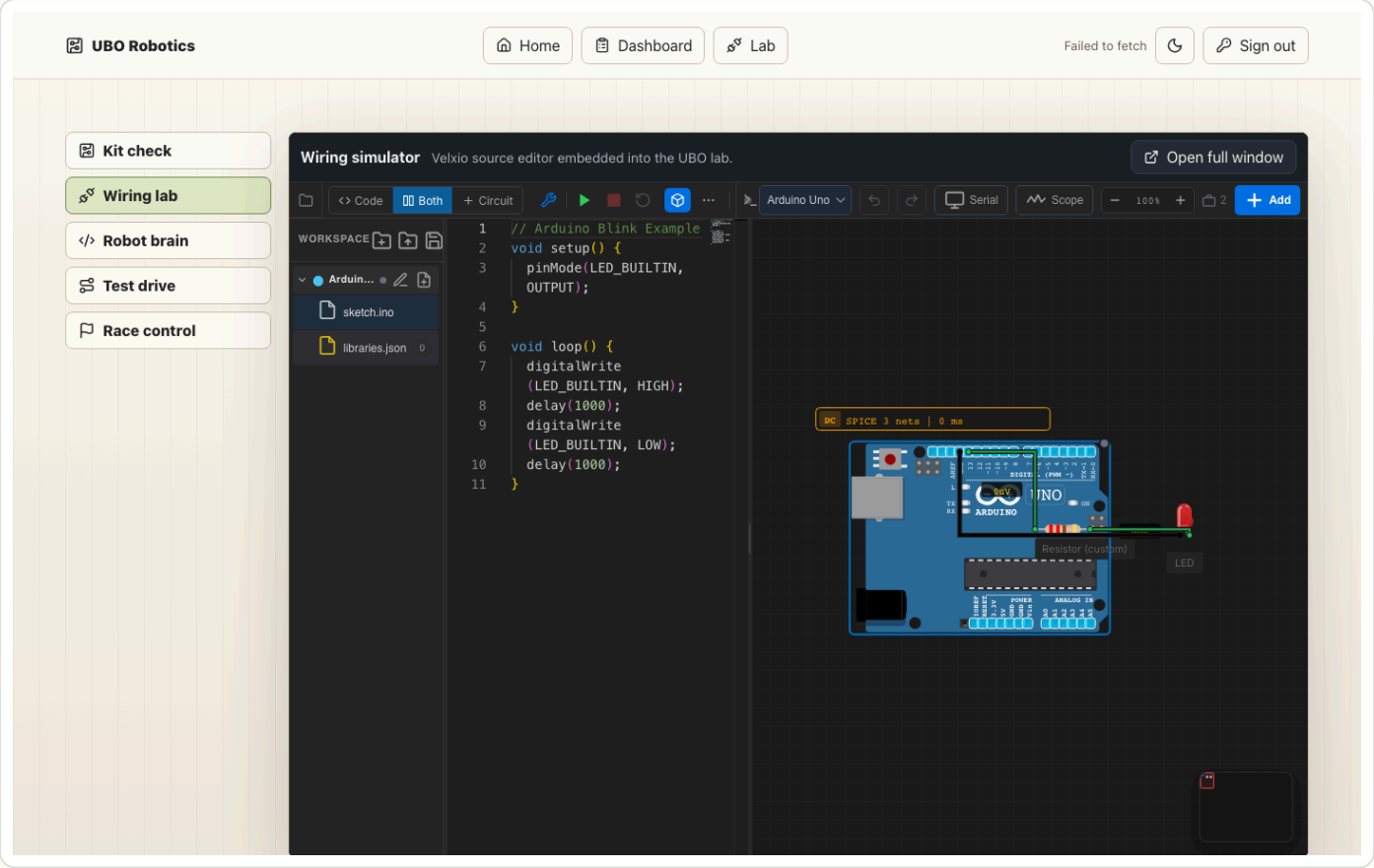

Figure 1: Open the UBO wiring editor.

Checkpoint

- `Wiring lab` is selected on the left.
- The embedded editor toolbar is visible.
- The canvas has the dark Wokwi-style grid.
- You can see `Code`, `Both`, `Circuit`, and `Run`.

2. Place The Parts Like A Workbench

Place the core parts in a readable left-to-right flow:

```
Arduino Uno -> L293D motor driver -> two motor loads
```

Use an `Arduino Uno`, an `L293D (Dual H-Bridge Motor Driver)`, and two `Resistor` parts set to `5 ohm`.

Beginner idea: why resistors are standing in for motors

The editor does not expose a DC motor part yet, so the two `5 ohm` resistors are acting as motor windings for this simulation. A real motor is not just a resistor, but it does have a coil of wire inside it. Current has to pass through that coil for the motor to create force. This resistor load lets you practice the motor-driver wiring pattern before the visual motor component exists.

Think of the top resistor as the left motor load and the bottom resistor as the right motor load.

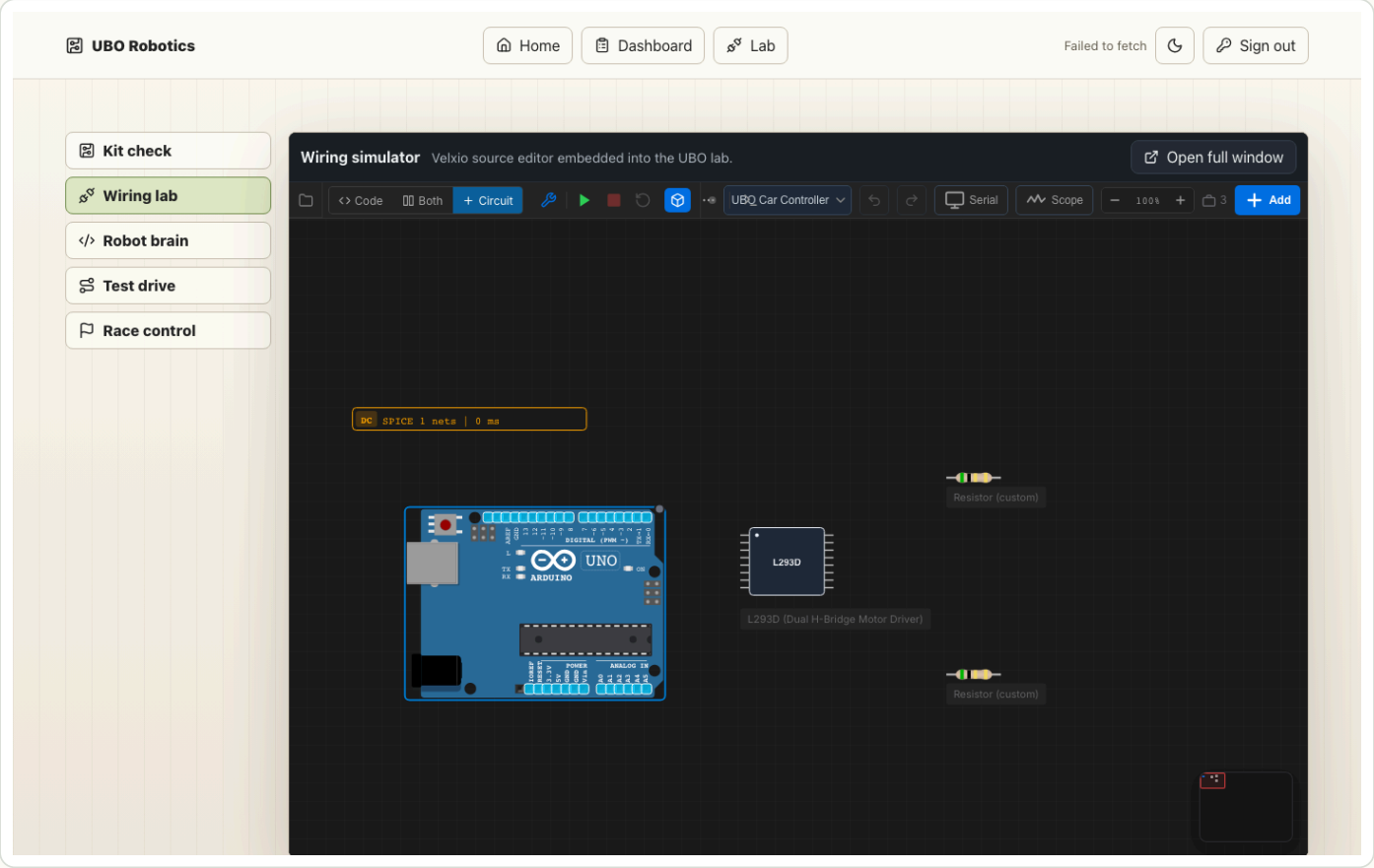

Figure 2: Place the Arduino, L293D, and two motor loads.

Do not wire everything yet. First, get the layout readable. Clean layout is part of debugging. If wires cross everywhere, it becomes harder to see whether a mistake is in the circuit or just in the drawing.

3. Wire Power And Shared Ground First

The L293D needs power before the Arduino direction pins mean anything. Wire the supply side first:

```
Arduino 5V  -> L293D VCC1
Arduino 5V  -> L293D VCC2
Arduino GND -> L293D GND.1
Arduino GND -> L293D GND.2
```

`VCC1` is logic power. It powers the part of the L293D that listens to Arduino control signals.

`VCC2` is motor-output power. It powers the side of the chip that drives the motor loads. In a physical robot, `VCC2` would usually come from a battery pack because motors need more current than a controller pin can safely provide. In this first simulator pass, using `5V` keeps the circuit simple.

Beginner idea: common ground

Ground is the zero-volt reference point. When the Arduino says a signal is `HIGH`, that only makes sense compared to ground. When it says a signal is `LOW`, that also means close to ground. If the Arduino and motor driver do not share ground, the driver may not understand the Arduino's signals correctly.

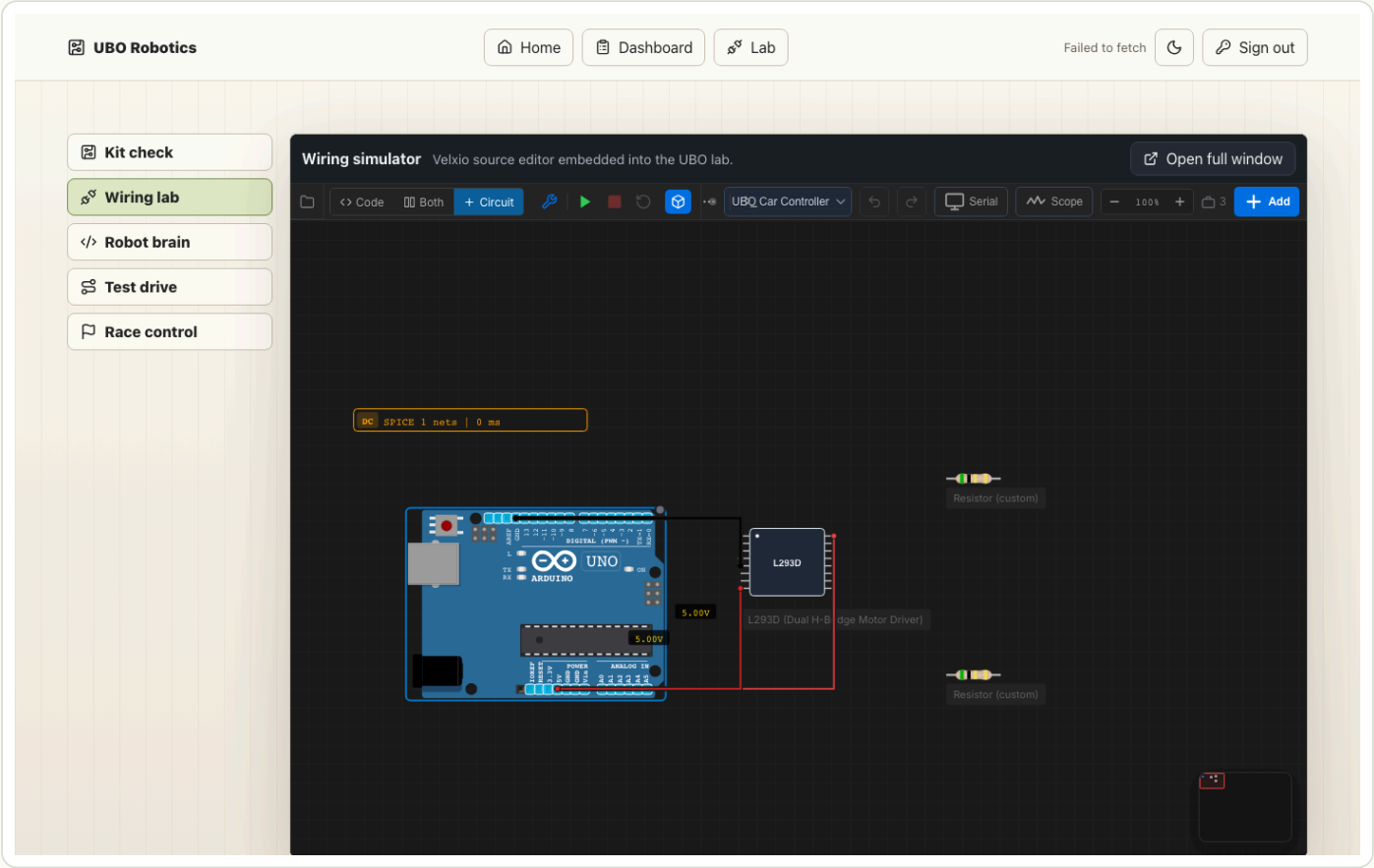

Figure 3: Wire 5V and shared ground.

Checkpoint

- `VCC1` has power.
- `VCC2` has power.
- Both L293D ground pins connect back to Arduino ground.
- The Arduino and driver share one electrical reference.

4. Wire Direction And PWM

Now connect the Arduino pins that control motion.

Arduino Pin	L293D Pin	Job
`D5`	`EN1`	Left motor speed with PWM
`D8`	`IN1`	Left motor direction A
`D7`	`IN2`	Left motor direction B
`D6`	`EN2`	Right motor speed with PWM
`D12`	`IN3`	Right motor direction A
`D11`	`IN4`	Right motor direction B

The `IN` pins are direction pins. They work in pairs. For one motor side, one input goes `HIGH` while the other goes `LOW`. That gives the driver a direction. Swap the pair, and the driver pushes current the opposite way.

```
IN1 = HIGH, IN2 = LOW  -> left motor forward
IN1 = LOW,  IN2 = HIGH -> left motor reverse
IN1 = LOW,  IN2 = LOW  -> left motor stop/coast
```

Beginner idea: PWM

PWM means pulse-width modulation. Instead of sending a halfway voltage, the Arduino switches the enable pin on and off very fast. More on-time feels like more power to the motor. Less on-time feels like less power. That is how one digital pin can control speed.

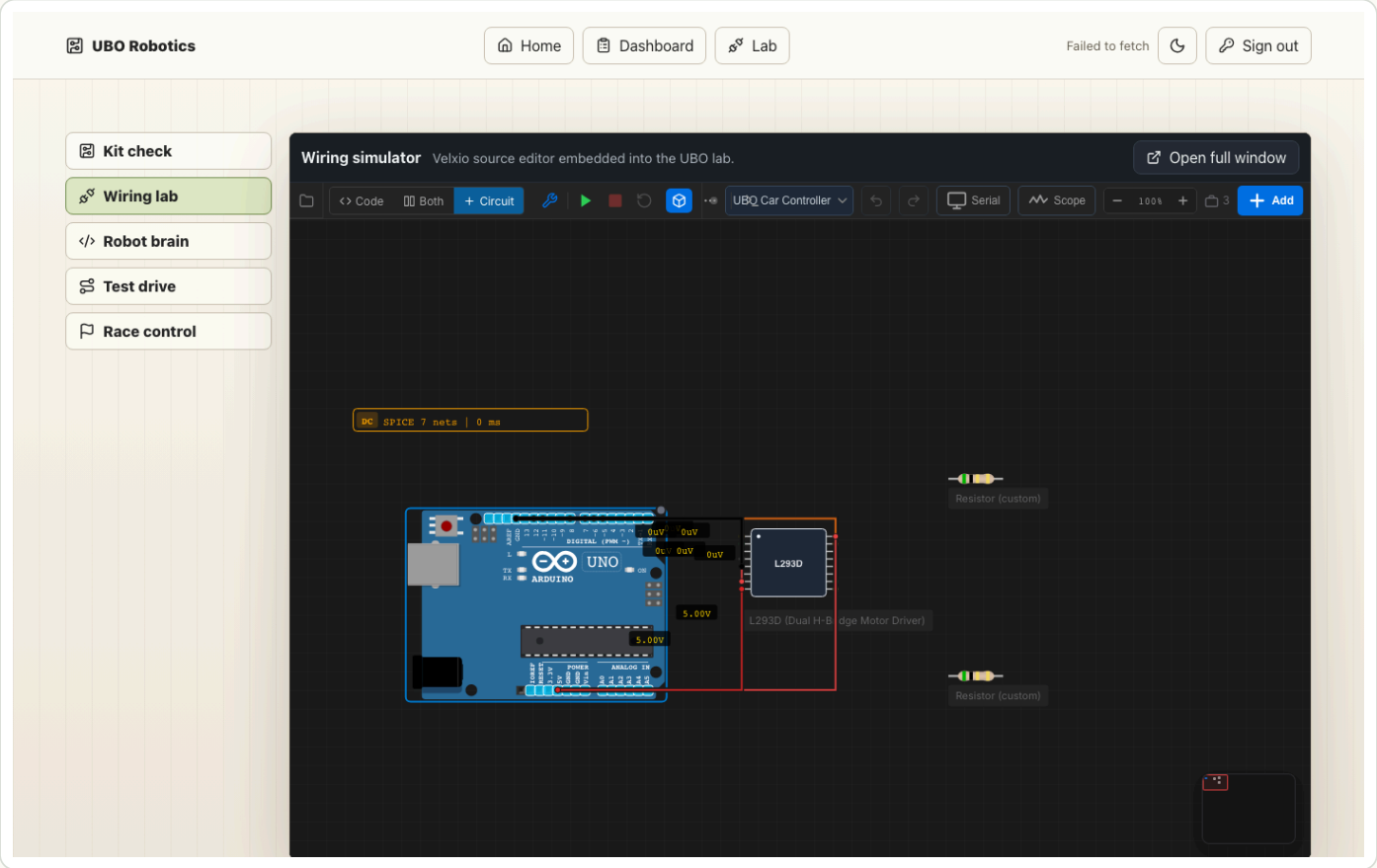

Figure 4: Wire direction pins and PWM speed pins.

This is where firmware and wiring lock together. If the code says the left enable pin is `D5`, then the wire really needs to go from `D5` to `EN1`.

5. Wire The Two Motor Loads

Now connect the load side of the L293D.

Output Pair	Load
`OUT1` and `OUT2`	Left motor load resistor
`OUT3` and `OUT4`	Right motor load resistor

This is the current path:

```
L293D output -> motor load resistor -> L293D output
```

Do not wire these motor loads straight to Arduino ground. The whole point of the H-bridge is that the driver controls which side of the load is high and which side is low. That is how it can reverse direction.

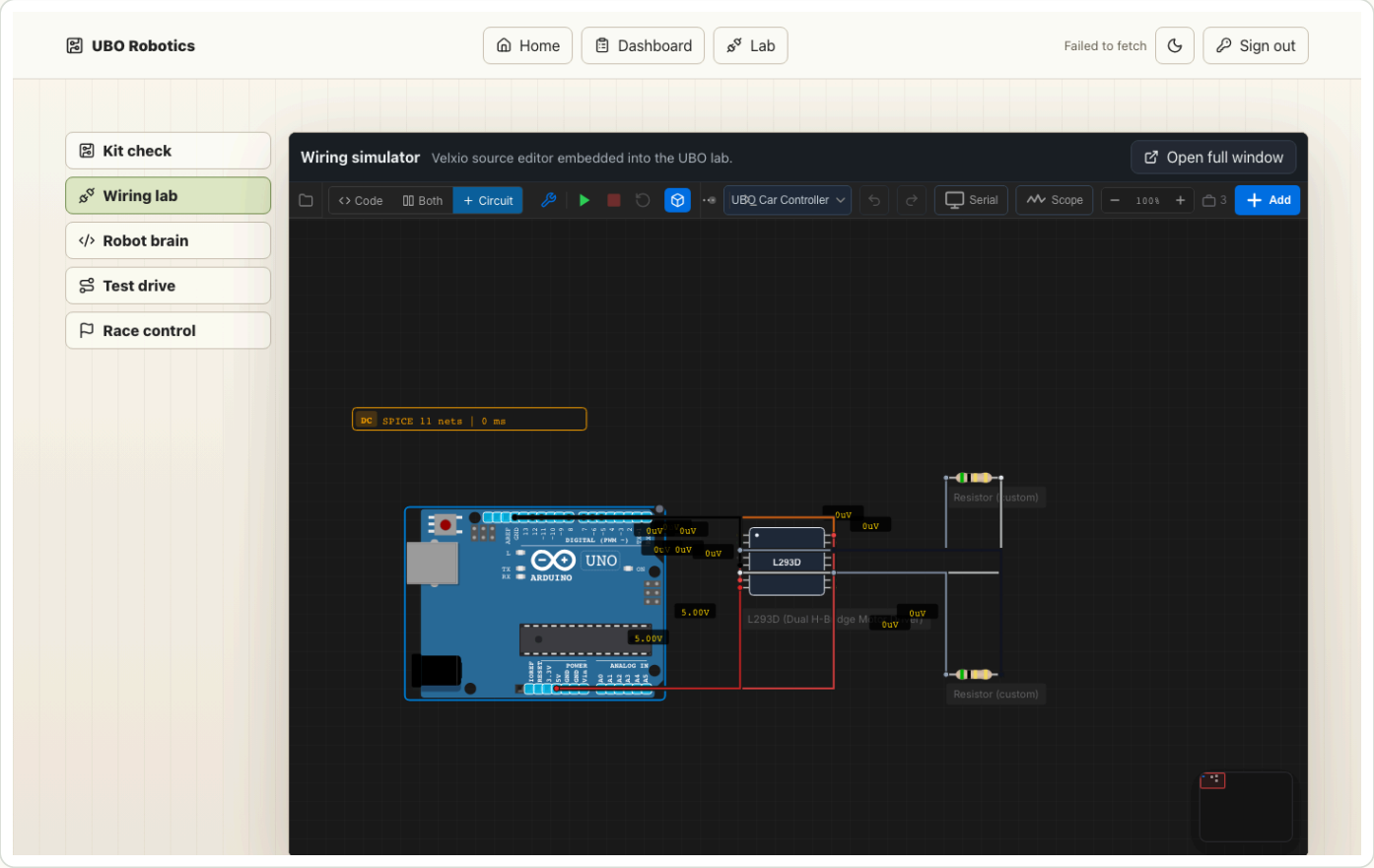

Figure 5: Wire the left and right motor loads.

If a motor goes backward

That is normal in robotics. Either swap that output pair or invert that side in firmware. A big part of real robot work is comparing expected behavior to measured behavior and correcting the map.

6. Paste The Firmware

Open `sketch.ino` and paste the firmware from `firmware/robot_car_drive.ino`.

The code is organized around behavior helpers:

```
driveForward()
driveBackward()
pivotRight()
pivotLeft()
stopRobot()
```

Each helper writes a direction pattern and a PWM speed. The firmware translates human driving language into electrical pin state.

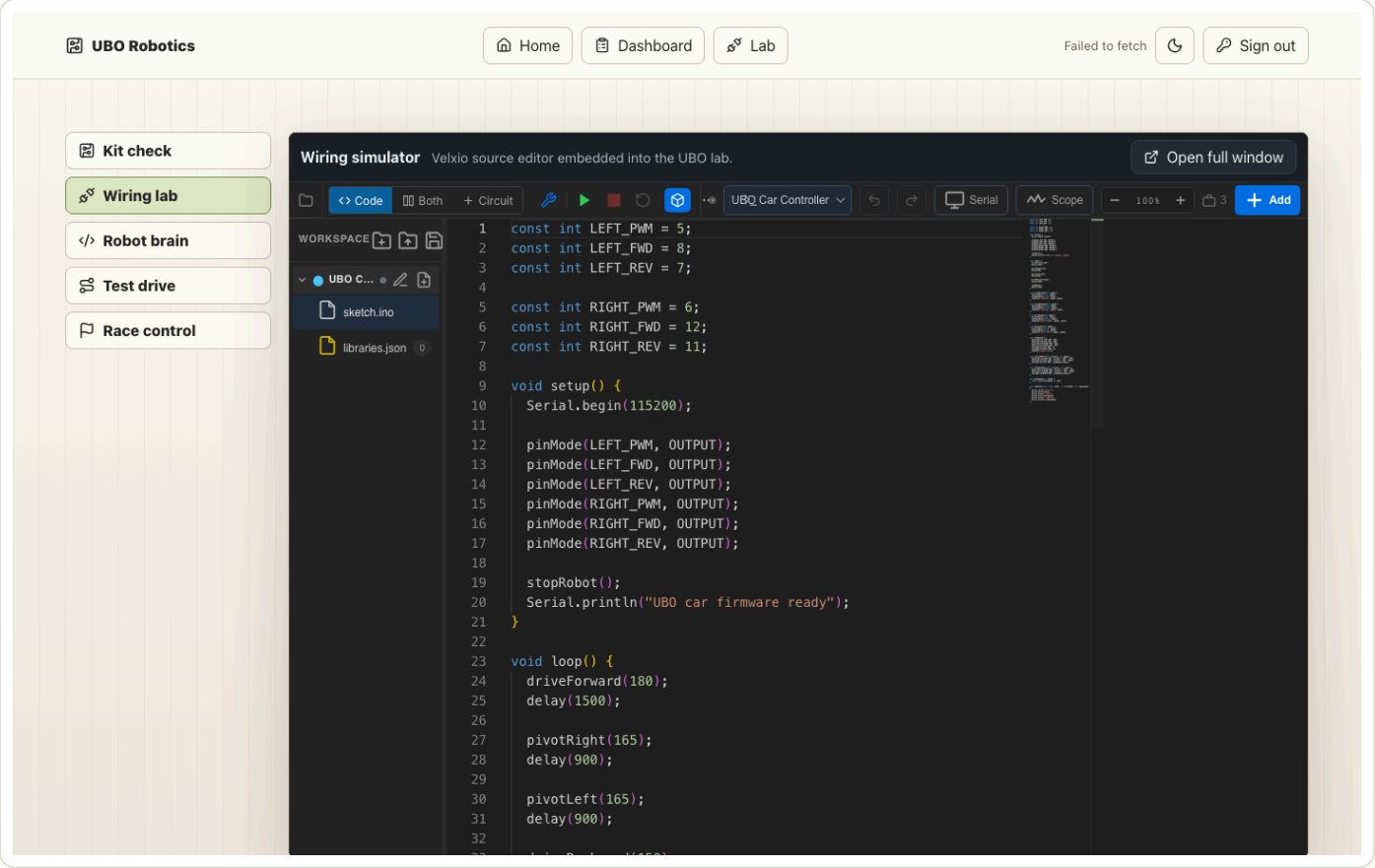

Figure 6: Paste robot car firmware into sketch.ino.

The key constants should match your wiring:

```
const int LEFT_PWM = 5;
const int LEFT_FWD = 8;
const int LEFT_REV = 7;
const int RIGHT_PWM = 6;
const int RIGHT_FWD = 12;
const int RIGHT_REV = 11;
```

When the code calls `digitalWrite(LEFT_FWD, HIGH)`, it tells the Arduino to push that pin toward `5V`. When it calls `digitalWrite(LEFT_REV, LOW)`, it pulls that pin toward `0V`. The L293D sees that pair and drives the left load in one direction.

7. Run And Read The Evidence

Click `Run`.

You are not just checking that the program starts. You are checking whether the serial story matches the circuit behavior.

Expected serial sequence:

```
UBO car firmware ready
state=forward left=180 right=180
state=pivot-right left=165 right=165
state=pivot-left left=165 right=165
state=reverse left=150 right=150
state=stop left=0 right=0
```

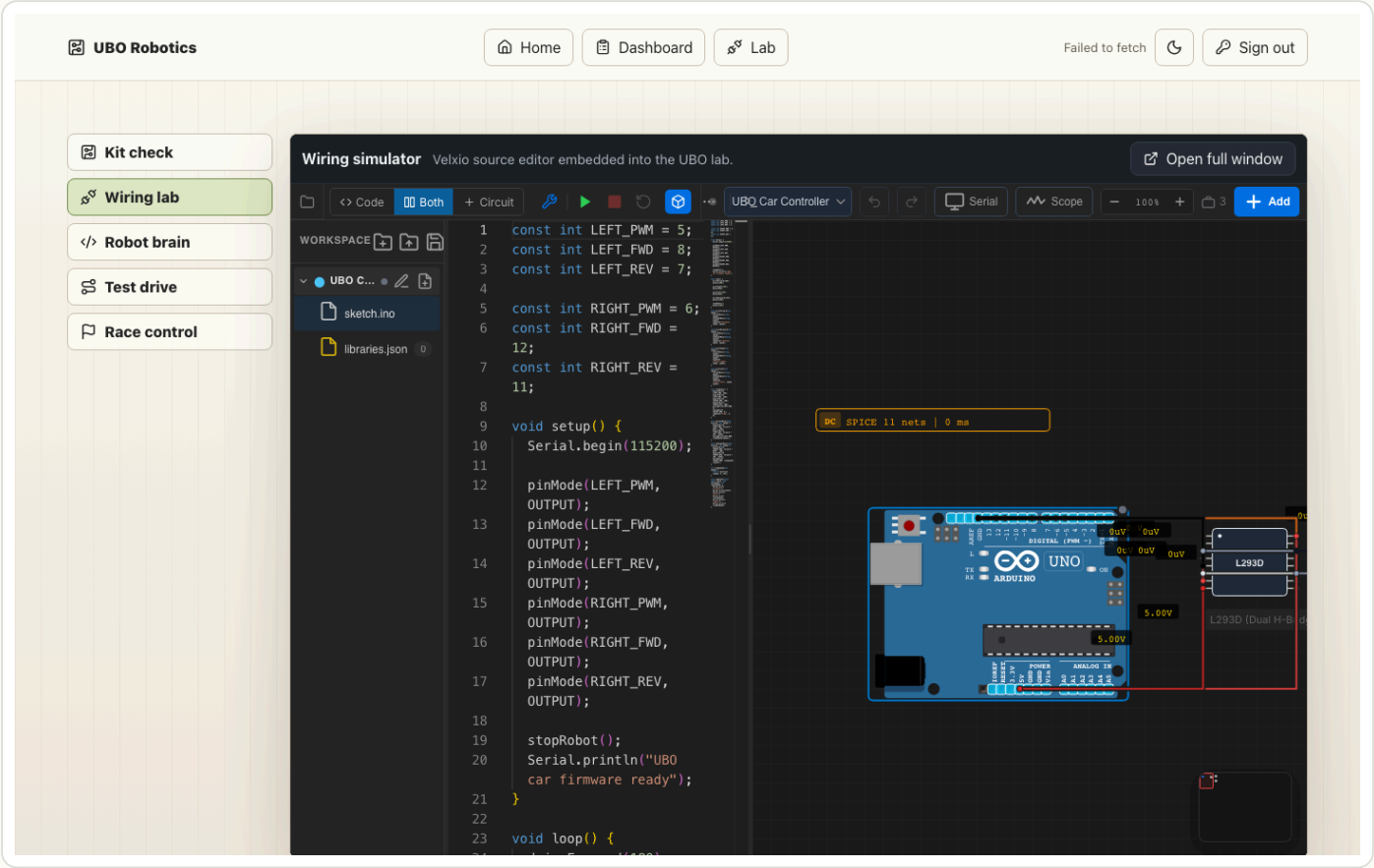

Figure 7: Run the firmware and compare behavior.

Checkpoint

- `forward`: both motor loads receive the forward direction pattern.
- `pivot-right`: left side forward, right side reverse.
- `pivot-left`: left side reverse, right side forward.
- `reverse`: both sides reverse.
- `stop`: direction pins LOW and PWM set to `0`.

If the serial output says `forward` but the wiring shows the wrong pins changing, trust the evidence. Debug the wire map before changing the code.

8. Connect The Circuit To Driving Controls

Switch to `Test drive`.

This is where the electronics lesson becomes a game/simulation lesson. The driving controls should call the same behavior helpers that the firmware used.

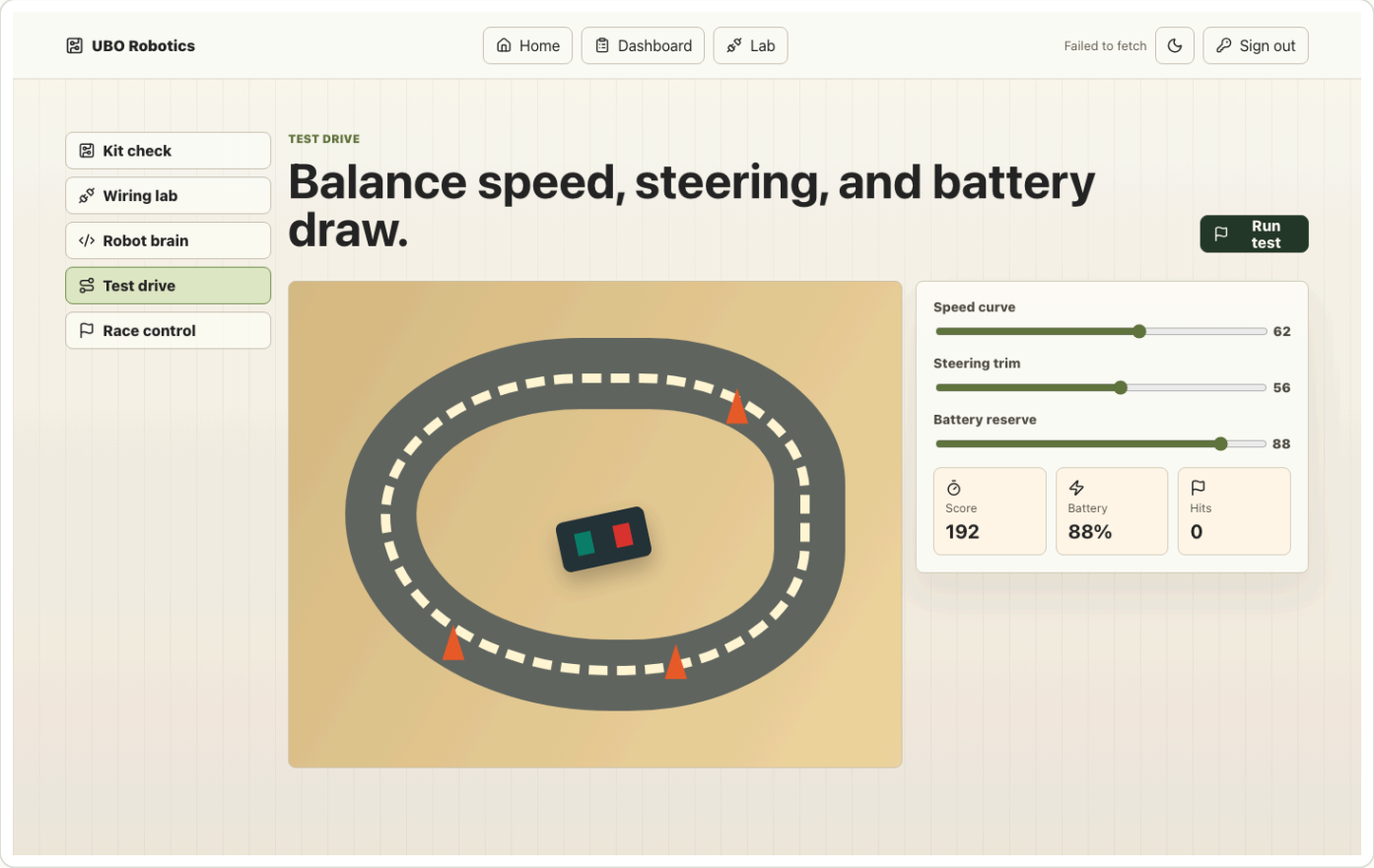

Figure 8: Open the test drive screen.

Control	Behavior
W	driveForward()
S	driveBackward()
A	pivotLeft()
D	pivotRight()
Space	stopRobot()

The game can consume motor state like this:

```
const robotState = {
  leftMotor: "forward",
  rightMotor: "reverse",
  leftSpeed: 165,
  rightSpeed: 165,
  action: "pivot-right"
};
```

If the left motor goes forward while the right motor goes backward, the robot pivots. The game does not need magic. It needs the same state your circuit already created.

9. Keep One Live Editor Reference

The final screenshot is the live editor after the staged project import. Use it as a reference if your canvas gets messy and you want to compare the overall layout.

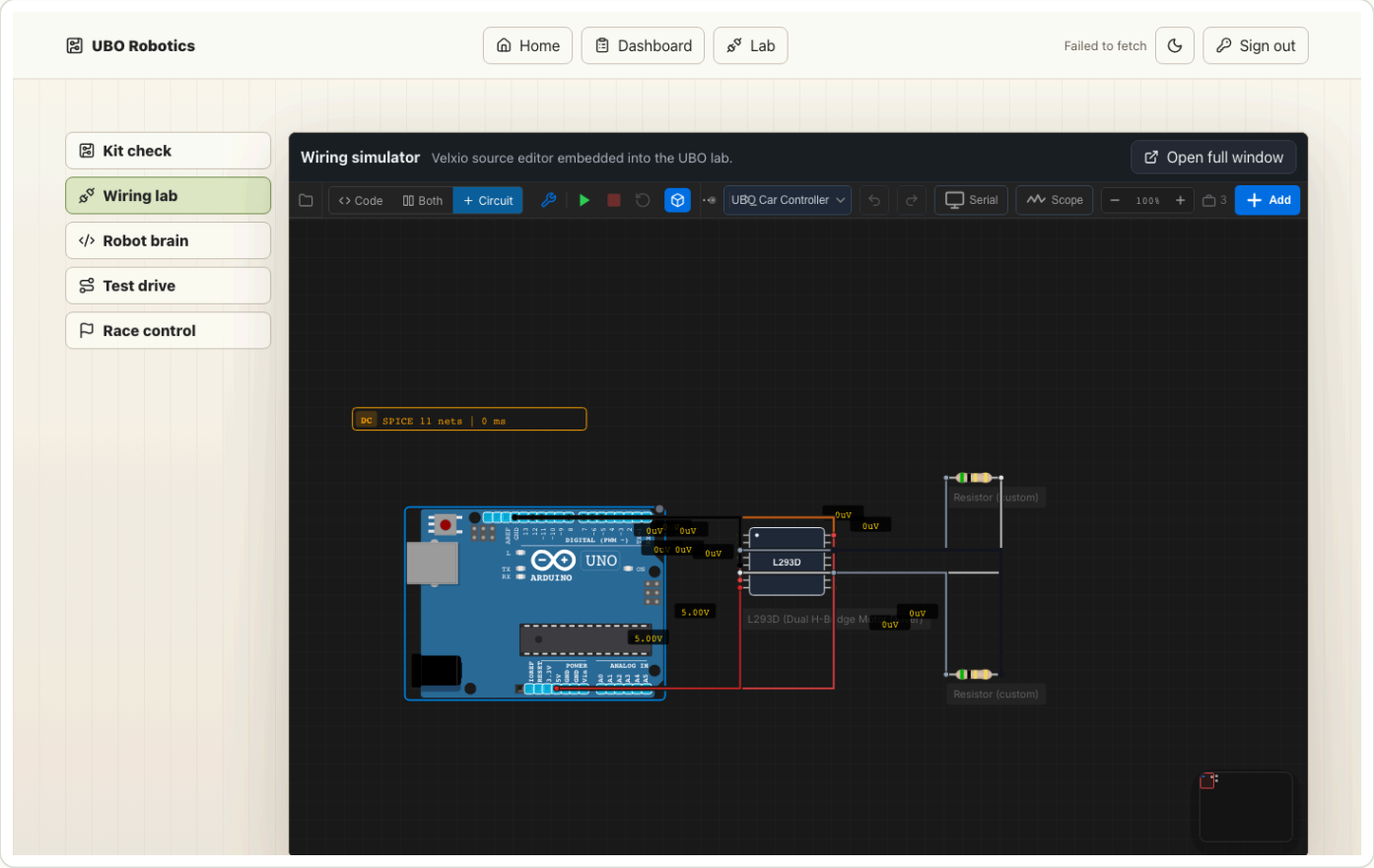

Figure 9: Live editor reference.

Debug Order

When something looks wrong, debug in this order:

1. Confirm both L293D ground pins share Arduino ground.
2. Confirm `VCC1` and `VCC2` are powered.
3. Confirm each motor-load resistor is wired between an output pair, not to ground.
4. Confirm direction pins match the firmware constants.
5. Confirm PWM pins are on `D5` and `D6`.
6. Read serial output before changing the wiring.

What You Learned

You built this chain:

```
Arduino pin state -> L293D output state -> left/right motor load behavior -> robot driving state
```

You also learned the beginner electronics ideas that make the chain work:

- `HIGH` means a pin is driven toward the board's positive voltage.
- `LOW` means a pin is driven toward ground.
- common ground gives parts the same zero-volt reference.
- PWM controls speed by switching power on and off quickly.
- a motor driver lets a small controller signal control a higher-current load.

The next lessons can swap in better motor visuals, sensors, line following, obstacle avoidance, telemetry, or race strategy, but the core loop stays the same: wire it, code it, run it, compare the evidence.